

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 11

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Race Condition Detection in Multithreaded Program

Question:

Develop a program using POSIX threads that demonstrates race condition. Multiple threads increment a shared counter without synchronization and detect the race condition.

Source Code:

```
#include <stdio.h>
#include <pthread.h>
#include <sched.h>

#define THREADS 4
#define ITERATIONS 1000000

volatile long long counter = 0;

void *increment(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        long long temp = counter;

        if (i % 1000 == 0)
            sched_yield();

        temp++;
        counter = temp;
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter : %d\n",
          THREADS * ITERATIONS);

    printf("Actual counter   : %lld\n", counter);

    if (counter != THREADS * ITERATIONS)
        printf("Race condition detected!\n");

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week11$ nano P11_T1.c
haasini@SSH:~/OSSP_upload/Practical_Week11$ gcc -Wall -Wextra P11_T1.c -o P11_T1 -pthread
haasini@SSH:~/OSSP_upload/Practical_Week11$ ./P11_T1
Expected counter : 40000000
Actual counter   : 1136489
Race condition detected!
haasini@SSH:~/OSSP_upload/Practical_Week11$ ./P11_T1
Expected counter : 40000000
Actual counter   : 1132970
Race condition detected!
```

Observations:

1. THREADS constant (4) defines the number of concurrent threads.
2. ITERATIONS constant (1000000) defines the number of increments per thread.
3. Volatile keyword tells compiler not to optimize counter access.
4. Expected counter value: $4 * 1000000 = 4000000$.
5. Actual counter value is significantly less due to race condition.
6. Race condition occurs when multiple threads access and modify shared data simultaneously.
7. The increment operation (counter++) is not atomic and involves load, increment, and store.
8. Lost updates occur when write operations overlap between threads.
9. sched_yield() allows context switching to increase race condition visibility.
10. Multiple runs show varying actual counter values demonstrating non-deterministic behavior.

Task 2: Mutex-Based Synchronization for Thread Safety

Question:

Modify the race condition program to use pthread_mutex_t for proper synchronization. Demonstrate that mutex locks prevent race conditions and ensure correct results.

Source Code:



haasini@SSH: ~/OSSP_upload



GNU nano 7.2

P11_T2.c *

```
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter : %d\n",
           THREADS * ITERATIONS);

    printf("Actual counter   : %lld\n", counter);

    if (counter == THREADS * ITERATIONS)
        printf("Synchronization successful.\n");

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week11$ nano P11_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week11$ gcc -Wall -Wextra P11_T2.c -o P11_T2 -pthread
haasini@SSH:~/OSSP_upload/Practical_Week11$ ./P11_T2
Expected counter : 4000000
Actual counter   : 4000000
Synchronization successful.
haasini@SSH:~/OSSP_upload/Practical_Week11$ |
```

Observations:

1. pthread_mutex_t lock ensures mutual exclusion of critical sections.
2. PTHREAD_MUTEX_INITIALIZER statically initializes the mutex.
3. pthread_mutex_lock() acquires the mutex before accessing shared data.
4. pthread_mutex_unlock() releases the mutex after modifying shared data.
5. With mutex protection, actual counter equals expected counter (4000000).
6. Mutex ensures that only one thread can execute the critical section at a time.
7. Counter increment within lock/unlock is atomically protected.
8. Synchronization successful message confirms proper mutual exclusion.
9. Performance may be lower than unsynchronized code due to lock contention.
10. Mutex-based synchronization guarantees correctness at the cost of some performance.